

CtxVigil — DA2 Review

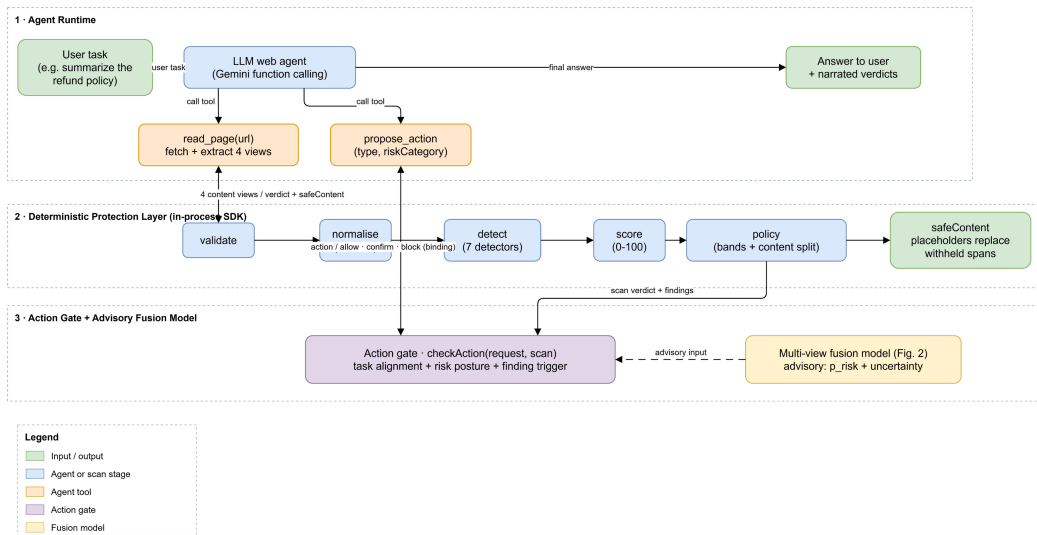
Proposed Project Architecture

What CtxVigil is

- ▶ A **protection layer between an LLM web agent and the web**: the agent never reads a page or performs an action directly — both pass through CtxVigil.
- ▶ Three cooperating components:
 1. **Deterministic scan pipeline** — validates, normalises a page into 4 content views, runs 7 rule detectors, scores 0–100, maps to `allow` / `sanitize` / `confirm` / `block` with withheld-content containment.
 2. **Independent action gate** — task alignment + risk-category posture + scan evidence decide `allow` / `confirm` / `block` for every proposed tool action.
 3. **Multi-view fusion model** (advisory) — MiniLM embeddings of 4 agent-context views + DeBERTa injection probability → risk + uncertainty heads.

Product: the `ctxvigil` npm SDK (`packages/ctxvigil-core`) + CLI + HTTP adapter + demo apps.

End-to-end application flow



Component 1 — Agent runtime

- ▶ A real LLM agent (Google **Gemini**, function-calling loop) with exactly two tools:
 - ▶ `read_page(url)` — fetches the page, extracts 4 content views, routes them through `scanPage`; the model receives only `safeContent` (placeholders replace withheld spans).
 - ▶ `propose_action(type, riskCategory)` — routes through the gate; its decision is final and binding.
- ▶ Enforcement lives in the loop, not the model's goodwill: on a block scan verdict the tool result is *never* returned to the model and the episode halts.

`apps/agent-chat/src/agent.ts` (loop, `haltOnBlock`) · `tools.ts` (`scanPage` / `checkAction` calls).

Component 2 — Scan pipeline (5 pure stages)

- ▶ **validate** — contract-exact request validation.
- ▶ **normalise** — page → ordered text segments across 4 content views: visible text, rendered DOM, hidden DOM (`display:none`, HTML comments), accessibility tree (ARIA); each segment keeps view, selector, and concealment metadata.
- ▶ **detect** — 7 deterministic detectors: instruction override, role impersonation, data exfiltration, risky action, hidden content, multi-view repetition, task conflict.
- ▶ **score** — category-weighted contributions → risk score 0–100 (task-aligned content damped).
- ▶ **policy** — score bands: 0–29 allow · 30–59 sanitize · 60–79 confirm · 80–100 block; partitions content into safe / sanitized / blocked.

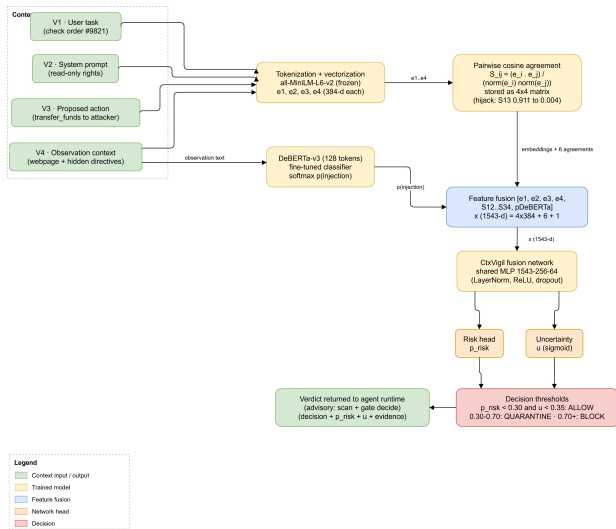
`packages/ctxvigil-core/src/` (validate, normalise, detect, score, policy) — all rule data in `src/config/defaults.ts`.

Component 3 — Action gate + advisory fusion model

- ▶ **Action gate** = an independent decision, never a re-run of the scan:
 - ▶ task alignment (action vs. the user's task),
 - ▶ risk-category posture (`account_change`, `data_transfer`, ...),
 - ▶ finding trigger (scan evidence forces a block).
- ▶ **Fusion model** (advisory): embeds V1 task / V2 system / V3 action / V4 observation with all-MiniLM-L6-v2, adds 6 cosine agreements + $p(\text{injection})$ from DeBERTa-v3 $\rightarrow$ 1,543-d $\rightarrow$ shared MLP $\rightarrow$ risk head + uncertainty head. The scan and gate remain the decisions of record.

`packages/ctxvigil-core/src/action/` · `src/multiview/` · Colab Notebook 2 (training).

Detailed data-flow of the layer



Data flow, input to output (one scan + one action)

1. User gives a task (*"summarize the refund policy"*).
2. Agent calls `read_page(url)`; the page is fetched and split into 4 content views.
3. Scan pipeline: views $\rightarrow$ findings (view, selector, signals) $\rightarrow$ risk score $\rightarrow$ verdict + `safeContent/sanitizedContent/blockedContent`.
4. The model sees only approved segments; a block halts the episode.
5. Agent calls `propose_action` for anything it wants to do.
6. Gate consumes action + scan evidence (+ fusion advisory) $\rightarrow$ `allow / confirm / block`.
7. Agent produces the final answer, narrating every verdict.

Technologies, frameworks, hardware, software

Protection layer (the product)

- ▶ TypeScript, Node.js; zero runtime dependencies
- ▶ Published as `ctxvigil` npm package
- ▶ Transports: in-process SDK, CLI, thin HTTP adapter
- ▶ Dashboard: React 19 + Vite
- ▶ Agent chat: Gemini API function calling
- ▶ Tests: 145 unit/contract tests, 7 fixture assertions

Learned models (Colab/Kaggle)

- ▶ PyTorch; Hugging Face Transformers
- ▶ `microsoft/deberta-v3-small` classifier
- ▶ `all-MiniLM-L6-v2` sentence embeddings
- ▶ scikit-learn (splits, metrics)
- ▶ Training hardware: Google Colab GPU (T4)
- ▶ Deployment runtime: commodity CPU, fully offline, no API key

`package.json` workspaces · notebooks 01/02 headers · apps/ (agent-chat, agent-demo, demo-web, protection-api).

Where every claim can be verified

- ▶ Scan pipeline code: `packages/ctxvigil-core/src/` (one folder per stage).
- ▶ All rule data (phrase tables, weights, bands, postures):
`src/config/defaults.ts`.
- ▶ Agent integration: `apps/agent-chat/src/tools.ts` — `guard.scanPage / guard.checkAction` calls.
- ▶ Model training: Colab Notebooks 1 and 2 (also in `notebooks/`).
- ▶ Live end-to-end demo: `npm run agent:demo` (asserts all six fixture verdicts offline).
- ▶ Diagram sources: `submission/report/figures/*.drawio` (editable in draw.io).